

Sistema De Gestion De Estudiantes

Juan Andrés Medina Pardo

Estudiante

William Alexander Matallana Porras

Docente

Universidad de Cundinamarca, Extensión Chía

17/02/2026

INTRODUCCIÓN

El desarrollo de software requiere una metodología estructurada que garantice organización, calidad y cumplimiento de objetivos. El Ciclo de Vida del Desarrollo de Software (SDLC) proporciona un marco de trabajo que permite planificar, analizar, diseñar e implementar sistemas de manera ordenada.

En el presente documento se describe el desarrollo de un Sistema de Gestión de Estudiantes, aplicado hasta la fase de codificación del SDLC. El sistema fue desarrollado en Java utilizando el patrón arquitectónico MVC (Modelo – Vista – Controlador), lo que permitió estructurar el código de manera modular, organizada y escalable.

El objetivo principal del sistema es permitir el registro, consulta, modificación y eliminación de estudiantes a través de una aplicación en consola, aplicando buenas prácticas de programación y principios de diseño estructurado.

JUSTIFICACIÓN

La gestión adecuada de información académica es fundamental en cualquier institución educativa. Un sistema organizado permite mejorar la eficiencia administrativa, reducir errores y optimizar el acceso a la información.

La implementación de este proyecto permite:

- Aplicar conocimientos de programación orientada a objetos.
- Comprender el uso práctico del modelo MVC.
- Aplicar las etapas del SDLC en un proyecto real.
- Desarrollar habilidades en estructuración y documentación de software.

Además, este proyecto fortalece la comprensión del proceso completo de desarrollo, desde la planificación hasta la implementación.

DESARROLLO DEL SDLC

1. Planificación (Planning)

En esta etapa se definió el objetivo del sistema: desarrollar una aplicación que permita gestionar estudiantes mediante operaciones básicas como registrar, listar, buscar, modificar y eliminar.

Se determinó el alcance del proyecto:

- Aplicación en consola.
- Almacenamiento en memoria (sin base de datos externa).
- Uso del lenguaje Java.
- Aplicación del patrón MVC.

Resultado: Documento de alcance y objetivos del sistema.

2. Análisis de Requerimientos (Requirements Analysis)

Se definieron los requerimientos funcionales:

- Registrar estudiante.
- Listar estudiantes.
- Buscar estudiante por ID.
- Modificar datos del estudiante.

- Eliminar estudiante.
- Generación automática de ID.

Requerimientos no funcionales:

- Código organizado y modular.
- Interfaz clara y comprensible.
- Cumplimiento de buenas prácticas de programación.

Resultado: Especificación detallada de funcionalidades.

3. Diseño del Sistema (System Design)

Se definió la arquitectura MVC:

Modelo (Model): Representa los datos del sistema.

Vista (View): Maneja la interacción con el usuario.

Controlador (Controller): Conecta la vista con la lógica del negocio.

Servicio (Service): Contiene la lógica de negocio.

Main: Punto de entrada del sistema.

Este diseño permite separación de responsabilidades y facilita el mantenimiento del sistema.

4. Diseño de Datos (Data Design)

Se estructuró la entidad Estudiante con los siguientes atributos:

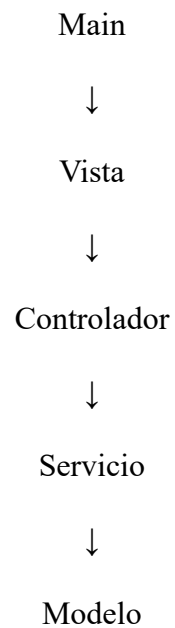
- ID (int)
- Nombre (String)
- Correo (String)
- Programa Académico (Enum)

Se utilizó una lista dinámica (List<Estudiante>) para almacenar los datos en memoria.

El uso de Enum garantiza que los programas académicos sean opciones predefinidas, evitando errores de digitación.

5. Diseño de Arquitectura (Architecture Design)

Flujo del sistema:



Este flujo garantiza que cada capa tenga una responsabilidad específica, evitando acoplamiento innecesario.

6. Diseño de Interfaz (UI Design)

Se diseñó una interfaz en consola con el siguiente menú:

1. Registrar estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Modificar estudiante
5. Eliminar estudiante
6. Salir

La interfaz incluye validaciones básicas y mensajes de confirmación para mejorar la experiencia del usuario.

7. Codificación (Implementation / Codification)

Se implementaron las siguientes clases:

- Estudiante: Representa la entidad principal.
- ProgramaAcademico: Enum con programas académicos.
- EstudianteService: Maneja la lógica de negocio.

- EstudianteController: Intermediario entre vista y servicio.
- EstudianteView: Maneja el menú e interacción.
- Main: Ejecuta la aplicación.

Durante la codificación se aplicaron principios de:

- Encapsulamiento.
- Modularidad.
- Separación de responsabilidades.
- Organización por paquetes.
- Uso de listas dinámicas.

El sistema final permite gestionar estudiantes correctamente desde la consola.

DIAGRAMA EXPLICATIVO DEL MVC (Representación Textual)

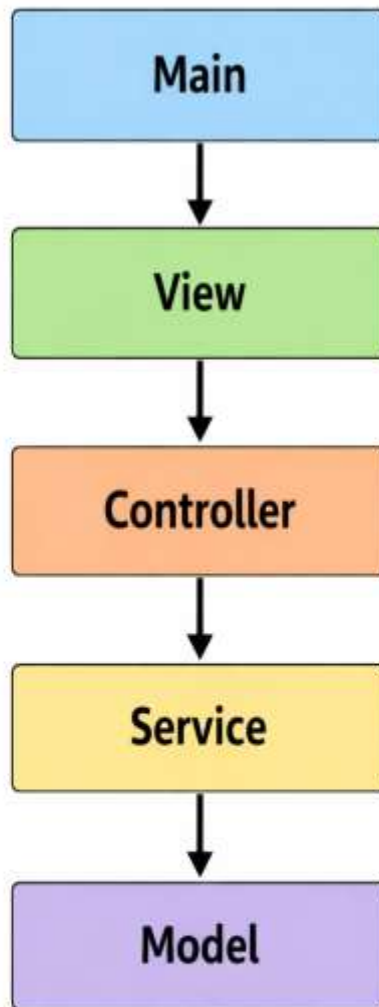


Diagrama casos de uso

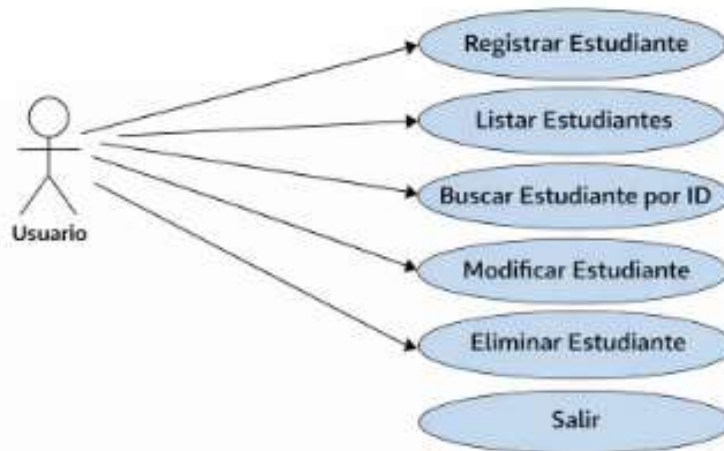
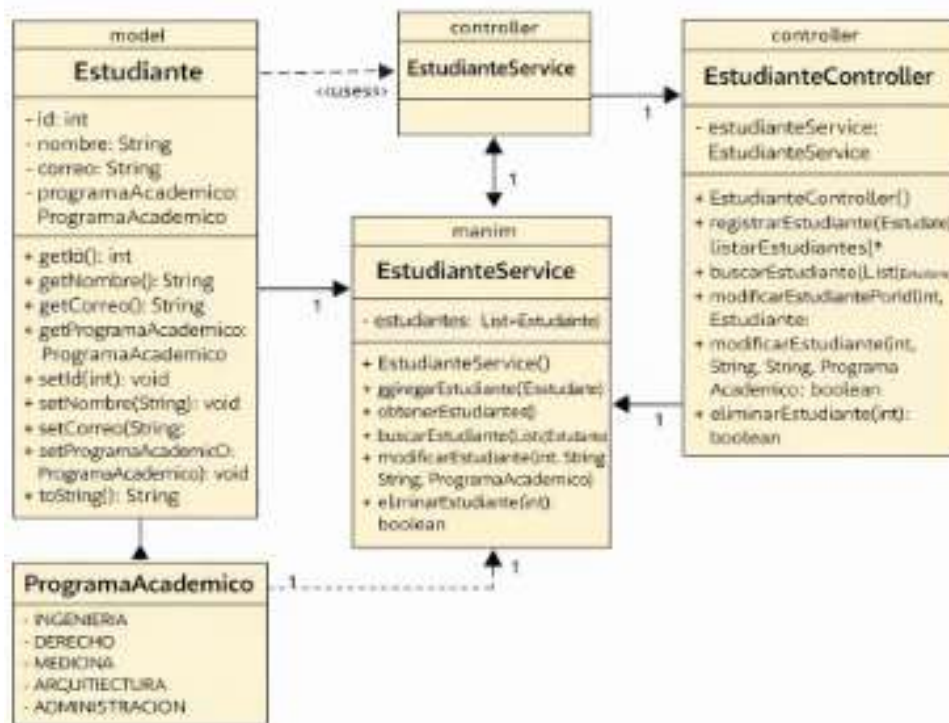


Diagrama de Casos de Uso



CONCLUSIONES

La aplicación del SDLC permitió desarrollar el sistema de manera estructurada y organizada. Cada etapa cumplió una función fundamental dentro del proceso de desarrollo.

El uso del patrón MVC permitió:

- Mejor organización del código.
- Mayor facilidad de mantenimiento.
- Separación clara de responsabilidades.
- Escalabilidad futura del sistema.

El proyecto demuestra la importancia de seguir metodologías formales en el desarrollo de software, incluso en proyectos académicos, ya que garantiza calidad y profesionalismo.

El sistema queda preparado para futuras etapas como pruebas formales, implementación en base de datos y despliegue.

REFERENCIAS BIBLIOGRÁFICAS

Sommerville, I. (2016). Ingeniería de software (10ª ed.). Pearson Educación.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Oracle. (2023). The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/>